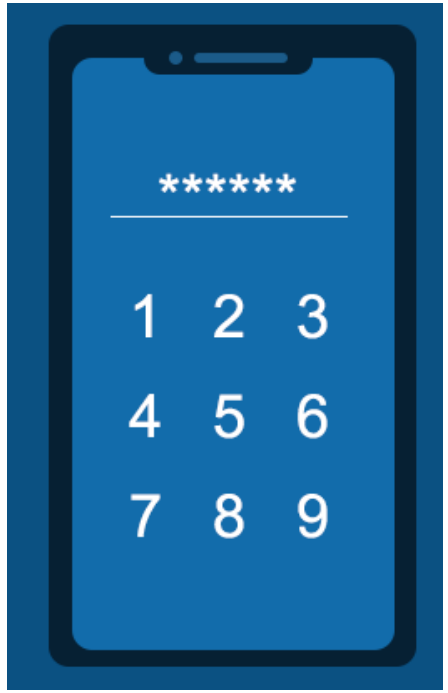
CSE 207
Digital Electronics

UNIT - I: Digital Fundamentals

Dr. S. V. K. Naresh
Assistant Professor
Department of Electrical and Electronics Engineering
naresh.s@srmap.edu.in

- What comes to your mind when I say "Digital Electronics"

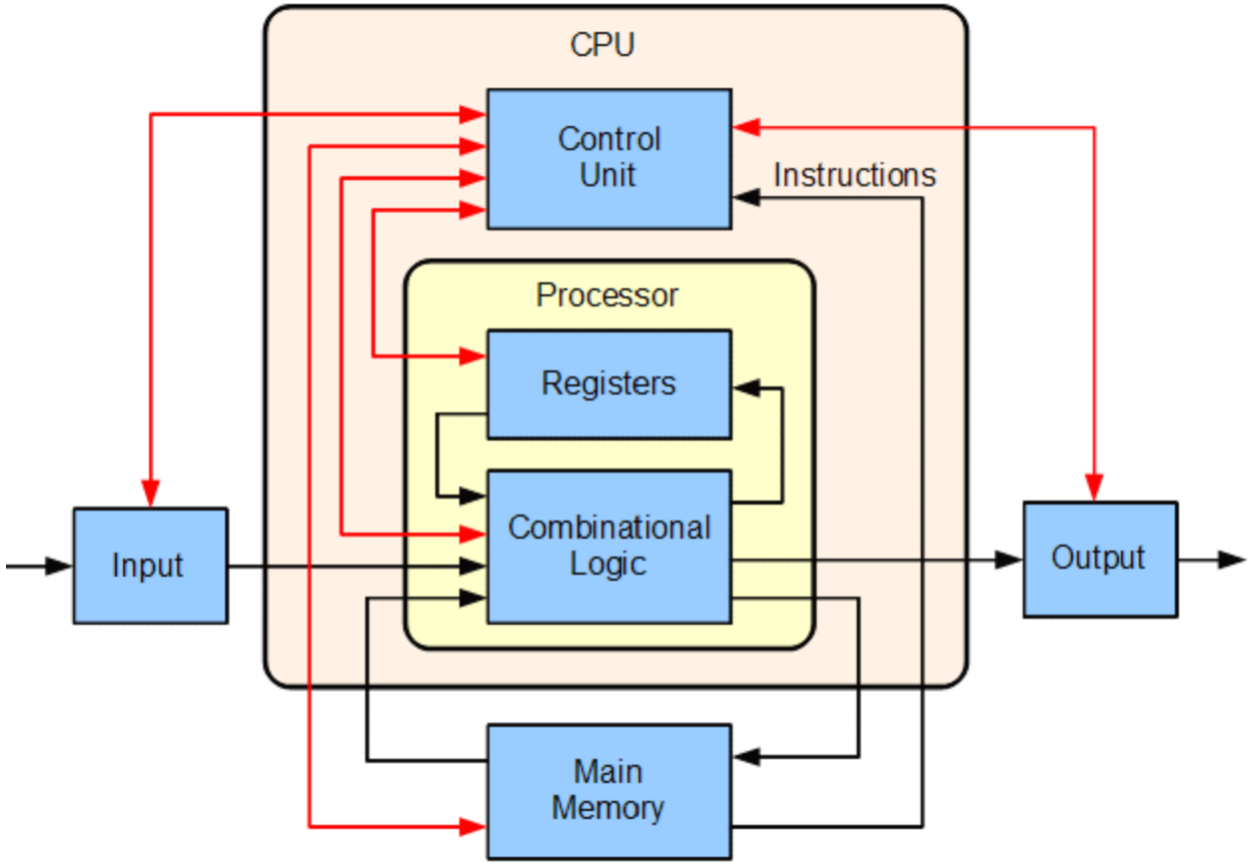
It deals with circuits that operate on binary values (0 and 1) to process, store, and transfer information.
Examples: Watches, Calculators, and Computers



- What is the first commercial product that adopted digital electronics?

Hamilton Pulsar
Digital watch
(1972)

➢ Why do you think it's important for CSE students to learn Digital Electronics?



- The computer operates with binary numbers
- For ALU operations, it requires adders, subtractors, and logic gates
- Programs: logics and decisions
- Controllers: Flip-flops, registers, counters
- Memory elements: ROM

"Digital Electronics" provides the background theory behind all the above functionalities.

Syllabus

- Unit 1: Digital Fundamentals
- Unit 2: Combinational Circuit Design
- Unit 3: Synchronous Sequential Circuits
- Unit 4: Registers and Counters
- Unit 5: Memory Devices

References:

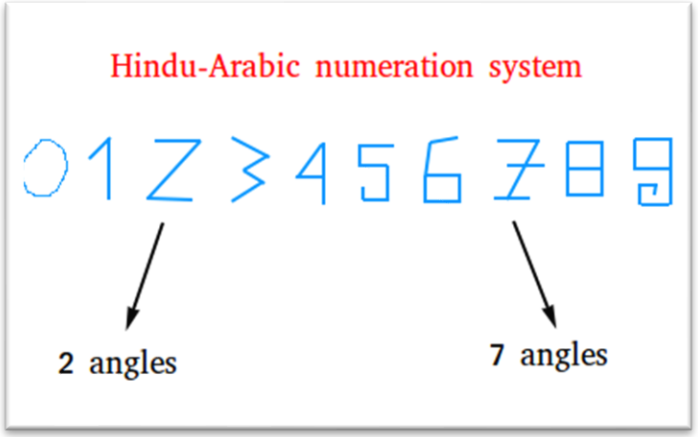
- NPTEL Lecture series "Switching Circuits and Logic Design" by Prof. Indranil Sengupta.

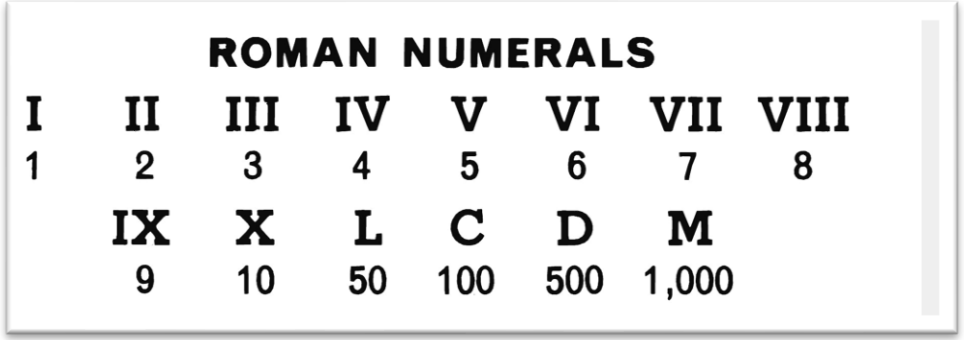
Unit 1: Digital Fundamentals

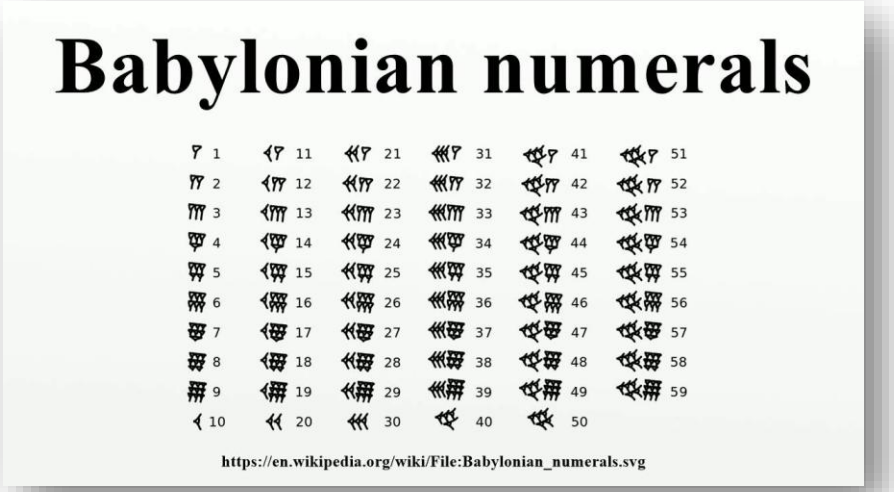
- Number Systems and Conversions
- Compliments
- Codes: Binary, BCD, Excess 3, Gray, Alphanumeric codes
- Logic gates
- Boolean Algebra
- Sum of products and product of sums
- K- Maps

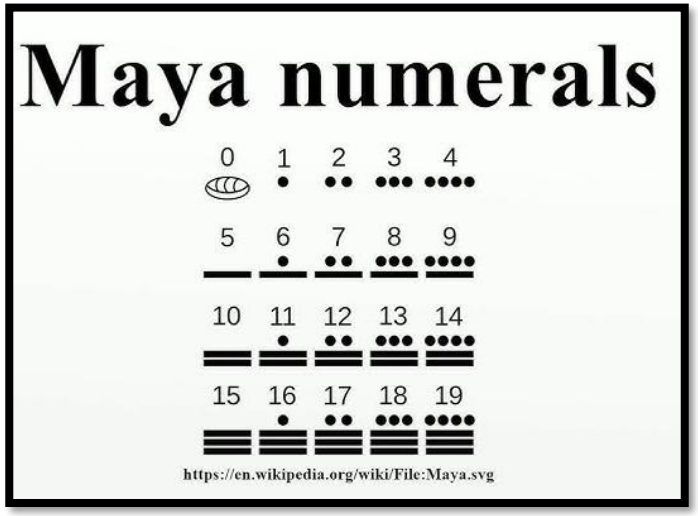
Number Systems

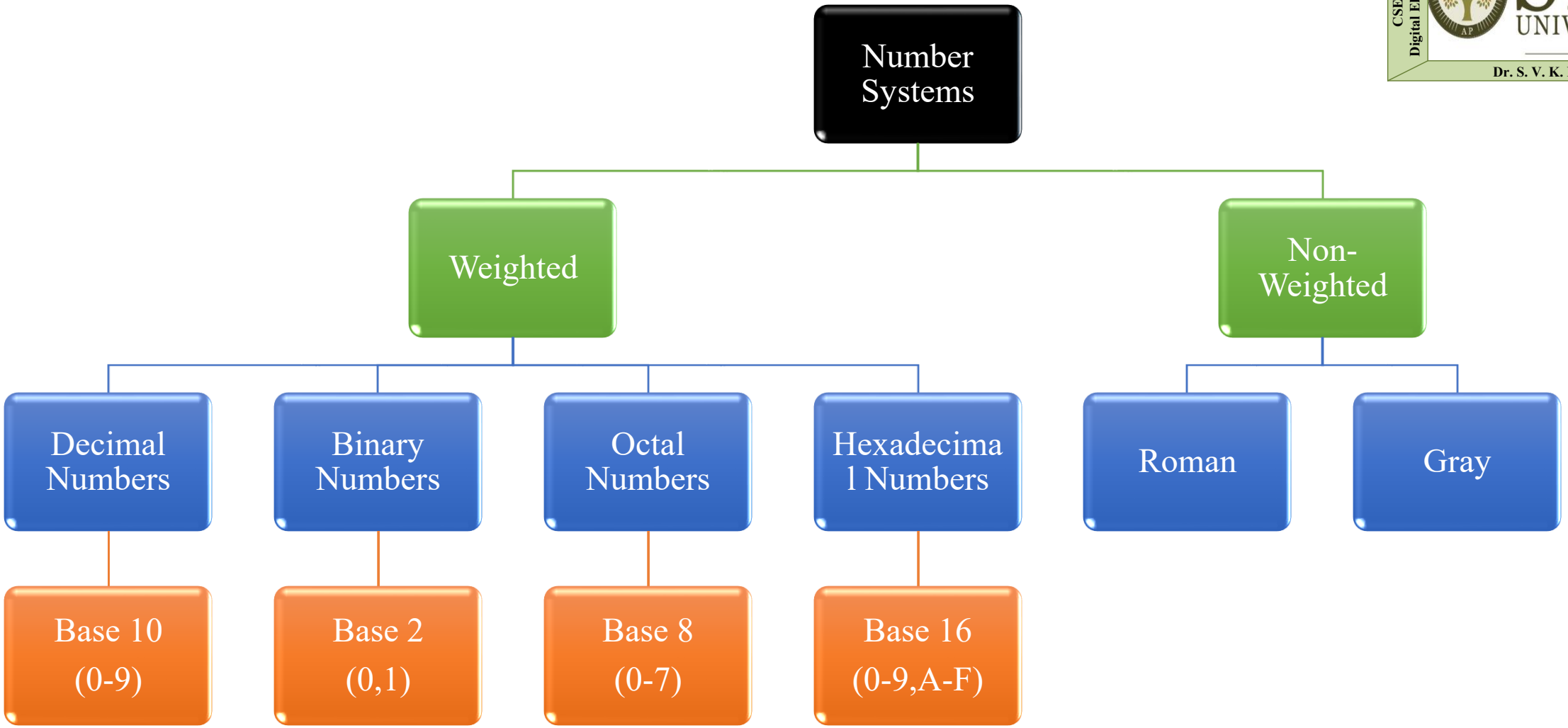
- Do you know any number system?










Number Systems
Weighted
Non-Weighted
Decimal Numbers
Binary Numbers
Octal Numbers
Hexadecimal Numbers
Roman
Gray
Base 10 (0-9)
Base 2 (0,1)
Base 8 (0-7)
Base 16 (0-9,A-F)

Weighted Number Systems

A (n+m) – digit number representation

$$D = d_{n-1}\, d_{n-2} \ldots d_1\, d_0 \,.\, d_{-1}\, d_{-2} \ldots\ldots d_{-m}$$

- **Base or Radix (r):** Number of distinct digits.
  - Assume that the digits are 0,1,2, - - - -, (r-1)
  - Every digit position has a weight that is some power of r (e.g., r^k)
    - ✓ $K \geq 0$, for the integer part
    - ✓ $K < 0$, for the fraction part

1. Decimal Number System:

- Base = 10
- Digits = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- Applications: Global use

Example:

1) $10718 = 1\times10^4 + 0\times10^3 + 7\times10^2 + 1\times10^1 + 8\times10^0$

2) $107.18 = 1\times10^2 + 0\times10^1 + 7\times10^0 + 1\times10^{-1} + 8\times10^{-2}$

2. Binary Number System:

- Base = 2
- Digits = 0, 1
- Applications: Digital devices

Example:

$(10.7)_{10} = 001\ 000.\ 111$

3. Octal Number System:

- Base = 8
- Digits = 0, 1, 2, 3, 4, 5, 6, 7.
- Applications: Legacy computing systems, Unix file permissions

Example:

$(17)_8 = 1\times8^1 + 7\times8^0 = (15)_{10}$

4. Hexa Decimal Number System:

- Base = 16
- Digits = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.
- Applications: Memory addresses, color codes in HTML/CSS

Example:

$(17)_8 = 1\times8^1 + 7\times8^0 = (15)_{10}$

Summary Number System:

Decimal	Binary	Octal	Hexadecimal
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

Number Conversions

1. Decimal to any Base:

Rule 1: To get the integer part, take ***successive divisions*** of the decimal integer part with the required base and ***accumulate the remainders in reverse order***.

Rule 2: To get the fractional part, take ***successive multiplication*** of the decimal fractional part with the required base and ***accumulate the integers in the original order***.

Conversion of $(107.18)_{10}$ to Base 2 (Binary):

Integer Part (Rule 1)

2	107
2	53 - 1
2	26 - 1
2	13 - 0
2	6 - 1
2	3 - 0
	1 - 1

1 1 0 1 0 1 1

Fractional Part (Rule 2)

$0.18 \times 2 = 0.36 \Rightarrow 0$

$0.36 \times 2 = 0.72 \Rightarrow 0$

$0.72 \times 2 = 1.44 \Rightarrow 1$

$0.44 \times 2 = 0.88 \Rightarrow 0$

$0.88 \times 2 = 1.76 \Rightarrow 1$

$0.76 \times 2 = 1.52 \Rightarrow 1$

0 0 1 0 1 1

$$(107.18)_{10} = (1101011.001011)_2$$

Conversion of $(107.18)_{10}$ to Base 8 (Octal):

Integer Part (Rule 1)

$$\begin{array}{r|l} 8 & 107 \\ \hline 8 & 13 - 3 \\ \hline & 1 - 5 \end{array}$$

1 5 3

Fractional Part (Rule 2)

$0.18 \times 8 = 1.44 \Rightarrow 1$

$0.44 \times 8 = 3.52 \Rightarrow 3$

$0.52 \times 8 = 4.16 \Rightarrow 4$

1 3 4

$$\mathbf{(107.18)_{10} = (153.134)_8}$$

Base 16 (Hexadecimal):

$$\mathbf{(107.18)_{10} = (6B.2E)_{16}}$$

Base 4 :

$$\mathbf{(107.18)_{10} = (1223.023)_4}$$

Number Conversions

2. Any Base to Decimal: Summation of Weighted Values

Let's verify the previous answers

1. $(1101011.001011)_2 = 1\times2^6 + 1\times2^5 + 0\times2^4 + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 + 0\times2^{-1} + 0\times2^{-2} + 1\times2^{-3} + 0\times2^{-4}$

$+1\times2^{-5} + 1\times2^{-6}$

$= 64 + 32 + 0 + 8 + 0 + 2 + 1 + 0 + 0 + 0.125 + 0.03125 + 0.015625$

$= (107.171875)_{10}$

2. $(153.134)_8 =$

3. $(6B.2E)_{16} =$

4. $(1223.0232)_4 =$

Example: Convert $(608.74)_9$ to Base 3?

1) *Any Base (9) → Decimal*
2) *Decimal → Any Base (3)*

1. Initially, convert Base 9 to Base 10 (Decimal) →

$$(608.74)_9 = 6\times9^2 + 0\times9^1 + 8\times9^0 + 7\times9^{-1} + 4\times9^{-2}$$
$$= 486 + 0 + 8 + 0.7777 + 0.0483827$$
$$= (494.8271605)_{10}$$

2. Convert Base 10 to Base 3:

3	494	
3	164	2
3	54	2
3	18	0
3	6	0
	2	0

2 0 0 0 2 2

$$(0.8271605)_{10} = (211011)_3$$

$$\boxed{(608.74)_9 = (494.82716)_{10} = (200022.211011)_3}$$

Number Conversions

3. Conversion of Base (b^n) to Base (b):

b^n → b ["n" digits *representation*]

Example: $(704.25)_8 = (\ )_2$

$8 = 2^3$ → 3 digits

= 111 000 100. 010 101

Convert $(608.74)_9$ to Base 3?

Base = 3 (0,1,2)

$9 = 3^2$ → 2 digits

Base 9	Base 3
0	00
1	01
2	02
3	10

Base 9	Base 3
4	11
5	12
6	20
7	21
8	22

$$(608.74)_9 = (200022.2111)_3$$

Convert $(3AF.C)_{16}$ to Base 2?

Base = 2 (0,1)

$16 = 2^4 \rightarrow$ 4 digits

3	A	F	.	C
0011	1010	1111	.	1100

Convert $(3AF.C)_{16}$ to Base 4?

Base = 4 (0,1,2,3)

$16 = 4^2 \rightarrow$ 2 digits

Base 16	Base 4
0	00
1	01
2	02
3	03
4	10
5	11
6	12
7	13

Base 16	Base 4
8	20
9	21
A	22
B	23
C	30
D	31
E	32
F	33

$$(3AF.C)_{16} = (032233.30)_4$$

Number Conversions

4. Conversion of Base (b) to Base (b^n):

$b \rightarrow b^n$ ("n" digits *grouping*)

Integer Part:

- Scan the number from ***right to left***
- ***Group the digits*** based on 'n' value
- Adding ***leading zeros***, if necessary

Fraction Part:

- Scan the number from ***left to right***
- ***Group the digits*** based on the 'n' value
- Adding ***trailing zeros***, if necessary

Convert the $(1011110.11011)_2$ to Base 8 and 16:

1. Base 8 (Octal): $8 = 2^3$, 3-digit grouping

001	011	110.	110	110
1	3	6 .	6	6

2. Base 16(Hexadecimal): $16 = 2^4$, 4-digit grouping

0101	1110.	1101	1000
5	E .	D	8

Convert the $(200022.2111)_3$ to Base 9:

Base 9: $9 = 3^2$, 2-digit grouping

20	00	22.	21	11
6	0	8 .	7	4

Base 9	Base 3
0	00
1	01
2	02
3	10
4	11
5	12
6	20
7	21
8	22

If 22 + 23 = 100, what is the base value of these numbers?

When the base is unknown, convert them to decimal numbers

Solution:

$(2\ 2)_b + (2\ 3)_b = (1\ 0\ 0)_b$

Converting to Decimal

$(2\ 2)_b = (\ )_{10} = 2*b^1 + 2*b^0 = 2b+2$
$(2\ 3)_b = (\ )_{10} = 2*b^1 + 3*b^0 = 2b+3$
$(1\ 0\ 0)_b = (\ )_{10} = 1*b^2 + 0*b^1 + 0*b^0 = b^2$

$(2b+2)_{10} + (2b+3)_{10} = (b^2)_{10}$

$2b+2+2b+3=b^2 \rightarrow \rightarrow b^2-4b-5=0$

$b = +5$ or $-1 \rightarrow b = 5$

** Base cannot be a negative value **

Verification:

- Substitute base = 5, in decimal conversion
  $2b+2 = 2*5+2 = 12$
  $2b+3 = 2*5+3 = 13$

$b^2 = 5*5 = 25$

Base 5 numbers:

B_{10}	B_5	B_{10}	B_5
0	00	5	10
1	01	6	11
2	02	7	12
3	03	8	13
4	04	9	14

```
  1
  2 2
  2 3
-------
1 0 0
```

If $\sqrt{41} = 5$, what is the base value of these numbers?

Answer : B = 6

If $\frac{312}{20} = 13.1$, what is the base value of these numbers?

Answer : B = 5

Binary Number Arithmetic's

1. Binary Addition:

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 0 *with Carry* 1.

Add 1101.101 and 111.011

```
  1  1  1  1     1  1
  1  1  0  1 .  1  0  1
  0  1  1  1 .  0  1  1
-------------------------
1  0  1  0  1 .  0  0  0
-------------------------
```

2. Binary Subtraction:

0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 *with Borrow* 1.

Subtract 111.111 from 1010.010

```
   1  10  1     1  10 10
1  0  1  0 .  0  1  0
0  1  1  1 .  1  1  1
-------------------------
0  0  1  0 .  0  1  1
-------------------------
```

Binary Number Arithmetic's

3. Binary Multiplication:

$0 \times 0 = 0$
$0 \times 1 = 0$
$1 \times 0 = 0$
$1 \times 1 = 1$

Multiply 1011.101 by 101.01

```
      1 0 1 1 . 1 0 1
                    ×
        1 0 1 . 0 1
      -------------
      1 0 1 1 1 0 1
    0 0 0 0 0 0 0 ×
  1 0 1 1 1 0 1 × ×
0 0 0 0 0 0 0 × × ×
1 0 1 1 1 0 1 × × × ×
---------------------
1 1 1 1 0 1 . 0 0 0 0 1
---------------------
```

Binary Number Arithmetic's

4. Binary Division:

Divide 101101 by 110

```
1 1 0 ) 1 0 1 1 0 1 ( 1 1 1 . 1
        1 1 0
        -------
        1 0 1 0
          1 1 0
        -------
          1 0 0 1
            1 1 0
          -------
            0 1 1 0
              1 1 0
            -------
              0 0 0
```

Divide 110101.11 by 101

```
1 0 1 ) 1 1 0 1 0 1 . 1 1 ( 1 0 1 0 . 1 1
        1 0 1
        -------
          0 1 1 0
            1 0 1
          -------
              1 1 1
              1 0 1
              -------
                1 0 1
                1 0 1
                -------
                  0 0 0
```

Complements

- **Radix complement [r's]**
- **Diminished radix complement [(r-1)'s]**

Assume a number N_r with n+m representation:

$$\mathbf{N_r = d_{n-1}\, d_{n-2} \ldots d_1\, d_0 \,.\, d_{-1}\, d_{-2} \ldots\ldots d_{-m}}$$

r's complement: $\mathbf{r^n - N}$

(r-1)'s complement: $\mathbf{r^n - r^{-m} - N}$

Find the r's and (r-1)'s complement of the numbers (i) $(10718)_{10}$ (ii) $(107.18)_{10}$

(i) $(10718)_{10}$ *r=10, n=5*

10's complement: r^n-N ➔ 10^5 – N

100000 – 10718 = 89282

9's complement: r^n-r^{-m}-N ➔ $10^5 - 10^{-0}$ - N

99999 – 10718 = 89281

Add +1 to LSB

(i) $(107.18)_{10}$ *r=10, n=3, m=2*

10's complement: r^n-N ➔ 10^3 – N

1000.00 – 107.18 = 892.82

9's complement: r^n-r^{-m}-N ➔ $10^3 - 10^{-2}$ - N

1000.00 – 0.01 – 107.18 = 892.81

Add +1 to LSB

Complements

Do (r-1)'s complement first, and then add +1 to the LSB.

Find the r's and (r-1)'s complement of the number $(307.206)_8$

Solution:

r=8, n=3, m=3

8's complement: r^n-N ➔ 8^3 – N

1000.000 – 307.206 = 470.572

7's complement: r^n-r^{-m}-N ➔ 8^3 – 8^{-3} - N

1000.000 – 0.001 – 307.206 = 470.571

Verification:

7's complement:

777.777 – 307.206 = 470.571

8's complement: *7's complement+1*

470.571 + 1 (LSB) = 470.572

Complements – Binary Numbers

- **1's complement** of a binary number is another binary number obtained by toggling all bits in it, i.e., *transforming the 0 bit to 1 and the 1 bit to 0*.

  Examples:
  1's complement of "0111" is "1000"
  1's complement of "1100" is "0011"

- **2's complement** of a binary number is *+1 added to the 1's complement* of the binary number.

  Examples:
  2's complement of "0111" is "(1000 +1 → 1001)"
  2's complement of "1100" is "(0011+1 → 0100)"

Sign Representation of Binary Numbers

Unsigned Binary Numbers:

Decimal	3-bit Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

- n-bit binary number → 2^n combinations

3-bit binary number → $2^3 = 8$ combinations
Range: 0 to 7 in decimal [0 to (2^n-1)]

4-bit binary number → $2^4 = 16$ combinations
Range: 0 to 15 in decimal [0 to (2^n-1)]

What about negative numbers?

Sign Representation of Binary Numbers

Signed Binary Numbers:

- Sign magnitude representation
- 1's complement representation
- 2's complement representation

Sign magnitude representation:

For an n-bit binary number representation:

- Most significant bit (MSB) indicates the sign (***0: positive & 1: negative***)
- Remaining (n-1) bits represent the magnitude of the number
- Range of numbers is $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$

Examples:

+4 → 0100	+7 → 0111	+0 → 0000
-4 → 1100	-7 → 1111	-0 → 1000

Two different representations for the zero

Sign Representation of Binary Numbers

4-bit Signed Number Representation

Decimal	4-bit Signed binary	Decimal	4-bit Signed binary
+0	0000	-0	1000
+1	0001	-1	1001
+2	0010	-2	1010
+3	0011	-3	1011
+4	0100	-4	1100
+5	0101	-5	1101
+6	0110	-6	1110
+7	0111	-7	1111

Sign Representation of Binary Numbers

1's complement representation:

For an n-bit binary number representation:

- Positive numbers are represented exactly as in sign-magnitude form
- Negative numbers are represented in 1's complement form
- Complement every bit of a number (0→1, 1→0)
- The most significant bit (MSB) indicates the sign (***0: positive & 1: negative***)
- The remaining (n-1) bits represent the magnitude of the number
- The range of numbers is $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$

Examples:

+5 → 0101	+7 → 0111	+0 → 0000
-5 → 1010	-7 → 1000	-0 → 1111

Two different representations for the zero

Sign Representation of Binary Numbers

4-bit Signed Number Representation

Decimal	1's Complement form	Decimal	1's Complement form
+0	0000	-7	1000
+1	0001	-6	1001
+2	0010	-5	1010
+3	0011	-4	1011
+4	0100	-3	1100
+5	0101	-2	1101
+6	0110	-1	1110
+7	0111	-0	1111

Sign Representation of Binary Numbers

2's complement representation:

For an n-bit binary number representation:

- Positive numbers are represented exactly as in sign-magnitude form
- Negative numbers are represented in 2's complement form
- Complement every bit of a number (0→1, 1→0), then add *+1 to the 1's complement*
- The most significant bit (MSB) indicates the sign (***0: positive & 1: negative***)
- The remaining (n-1) bits represent the magnitude of the number
- The range of numbers is $-(2^{n-1})$ to $+(2^{n-1}-1)$

Examples:

+6 → 0110	+7 → 0111	+0 → 0000
-6 → 1010	-7 → 1001	-0 → 0000

Single representations for the zero

Every computer uses the 2's complement representation form for storing negative numbers

Sign Representation of Binary Numbers

4-bit Signed Number Representation

Decimal	2's Complement form	Decimal	2's Complement form
+0	0000	-8	1000
+1	0001	-7	1001
+2	0010	-6	1010
+3	0011	-5	1011
+4	0100	-4	1100
+5	0101	-3	1101
+6	0110	-2	1110
+7	0111	-1	1111

Complements – Binary Numbers

Three ways of obtaining the 2's complement of a number (-45):

- **First Method: 1's complement + 1 to LSB**

Example: Binary number of +45 = 00101101
1's complement of +45 = 11010010
Add 1 = 11010011 (*2's complement form of -45*)

- **Second Method: Subtract the given number from 2^n**

Example: 2^n = 100000000
subtract binary number of +45 = 00101101
result = 11010011 (*2's complement form of -45*)

- **Third Method: Copy up to the first 1 bit from the LSB, and then complement the remaining bits**

Example: original number of +45 = 0 0 1 0 1 1 0 1
copy up to first 1 bit = 1
complement remaining bits = 1 1 0 1 0 0 1
result = 1 1 0 1 0 0 1 1 (*2's complement form of -45*)

Features of 2's complement representation:

1. Weighted number representation, with MSB having a weight -2^{n-1}

Examples:

- $0101 = -2^3\times0+2^2\times1+2^1\times0+2^0\times1 = 0+4+0+1 = +5$
- $1101 = -2^3\times1+2^2\times1+2^1\times0+2^0\times1 = -8+4+0+1 = -3$

2. The sign bit can be copied as many times as required in the beginning to extend the size of the number (sign extension)

Examples:

- +45 = 00101101 (8-bit representation) ➔ sign extension to 16-bit representation

  0000 0000 0010 1101 = +45

- -45 = 11010011 (8-bit representation) ➔ sign extension to 16-bit representation

  1111 1111 1101 0011 = -45

Features of 2's complement representation:

3. Shift left by 'k' positions with zero padding multiplies the number by '2^k'

Examples:

- 00010010 = +18 → Shift left by 2 bits ➔ 01001**00** = 64+4 = +72 [18×4]
- 11101110 = -18 → Shift left by 2 bits ➔ 10111**00** = -128+32+16+8 = -72 [-18×4]

4. Shift right by 'k' positions with sign-bit padding divides the number by '2^k'

Examples:

- ✓ +20 = 00010100 → Shift right by 2 sign bits ➔ **00**000101 = +5 [+20/4 = 5]
- ✓ -28 = 11100100 → Shift right by 2 sign bits ➔ **11**111001 = -7 [+28/4 = 7]

2's Complement Arithmetic:

Examples: Using the 2's complements, find the sum of the following 8-bit binary numbers

1. Add +6 and +13
2. Add -6 and +13
3. Add +6 and -13
4. Add -6 and -13

```
+6  = 0 0 0 0 0 1 1 0
+13 = 0 0 0 0 1 1 0 1
-----------------------
+19 = 0 0 0 1 0 0 1 1
```

```
- 6  = 1 1 1 1 1 0 1 0 (2's complement form)
+13  = 0 0 0 0 1 1 0 1
-----------------------
+7   = 0 0 0 0 0 1 1 1  (Discard the carry)
```

```
+ 6  = 0 0 0 0 0 1 1 0
-13   = 1 1 1 1 0 0 1 1 (2's complement form)
-----------------------
  -7  = 1 1 1 1 1 0 0 1
```

```
- 6  = 1 1 1 1 1 0 1 0 (2's complement form)
-13   = 1 1 1 1 0 0 1 1 (2's complement form)
-----------------------
-19   = 1 1 1 0 1 1 0 1
```

Examples on 2's complement

- Gate2025-1 (Q25): The number −6 can be represented as 1010 in 4-bit 2's complement representation. Which of the following is/are the CORRECT 2's complement representation(s) of −6? :

(A) 1000 1010 in 8-bits
(B) 1111 1010 in 8-bits
(C) 1000 0000 0000 1010 in 16-bits
(D) 1111 1111 1111 1010 in 16-bits

Answer: (b, d)

Binary Codes

Advantages of Binary Code:

→ Binary codes are suitable for computer applications.

→ Binary codes are suitable for digital communications.

→ Binary codes are used in the analysis and design of digital circuits.

→ Since only 0 & 1 are being used, implementation becomes easy.

Different types of Binary Code:

➔ Binary coded decimal (BCD)

➔ Excess-3 code

➔ Gray code.

➔ Alphanumeric codes (ASCII code).

➔ Error detection and correction code.

Binary Coded Decimal (BCD)

Perform binary addition:

- ✓ Convert 5 and 13 to binary
- ✓ Binary addition
- ✓ Binary to decimal conversion to display 18.

Can we eliminate conversions for simple arithmetic's

- ❑ Humans use decimal numbers
- ❑ Machines use binary numbers
- ❑ Computers are needed for the conversion of decimal to binary and vice versa.

Can we have a simple method: **Binary Coded Decimal (BCD)**

Binary Coded Decimal (BCD)

Each decimal digit is represented by its 4-bit binary equivalent, so that conversions can be easier.

Decimal digit	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Applications:

→ Display Devices (watches, calculators)
→ Older computers (8051 microcontroller)

Advantages:

→ Easy arithmetic's
→ Can be used in error detection

Disadvantages:

→ needed more space (59 in 8-bit of BCD, 6-bit in binary)
→ slower

1010, 1011, 1100, 1101, 1110, 1111 – invalid in BCD.

Binary Coded Decimal (BCD) – Addition:

- **BCD addition requires a correction step, where 6 (0110) is added to a nibble (4 bits)**

  → **If nibble is an invalid number**
  → **If there is carry**

07 -	0	0	0	0	0	1	1	1	
03 -	0	0	0	0	0	0	1	1	
	0	0	0	0	1	0	1	0	(Invalid Nibble)
				+	0	1	1	0	
10 -	0	0	0	1	0	0	0	0	

				1					
19 -	0	0	0	1	1	0	0	1	
18 -	0	0	0	1	1	0	0	0	
	0	0	1	1	0	0	0	1	Carry generated
				+	0	1	1	0	
37 -	0	0	1	0	0	1	0	1	

Excess-3 Code

- Excess-3 (XS-3) is a non-weighted binary-coded decimal (BCD) code.
- It is obtained by adding 3 (0011) to each decimal digit and then writing the result in 4-bit binary.

Decimal	BCD (8421)	Excess-3 (Add 0011)
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Significance of Excess-3 Code

1. Self-Complementing Property

- 9's complement can be obtained by simply *inverting all bits* in the Excess-3 code.

2. Error Detection

- Since Excess-3 skips invalid patterns (0000, 0001, 0010, etc.), it provides *built-in error detection* capability for certain codes.

3. Used in Old Digital Systems

- Popular in *decimal arithmetic units* of older computers and calculators.

4. No "all 0's" Representation

- Unlike BCD, where 0 = 0000, Excess-3's smallest code is 0011.
- This avoids *confusion with idle states* (all zeros in circuits).

Gray Code

- Gray code is a non-weighted binary code where successive code words differ in only one bit (cyclic code).
- It is a self-reflection code.

Decimal	Binary	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

One-bit Gray code	Two-bit Gray code		Three-bit Gray code		Four-bit Gray code
0	0	00	00	000	000	0000
1	1	01	01	001	001	0001
	1	11	11	011	011	0011
	0	10	10	010	010	0010
			10	110	110	0110
			11	111	111	0111
			01	101	101	0101
			00	100	100	0100
					100	1100
					101	1101
					111	1111
					110	1110
					010	1010
					011	1011
					001	1001
					000	1000

Gray Code

Why is Gray Code Important?

- **Error Minimization in Digital Systems**
  - → In binary, multiple bits may change at once (e.g., 0111 → 1000 flips 4 bits).
  - → In Gray code, only one bit changes at a time, reducing error during transitions.
- **Used in Mechanical and Digital Encoders**
  - → Rotary and shaft encoders use Gray Code so that if the sensor is between two positions, only 1-bit error occurs, not multiple.
- **Simplifies Circuit Design**
  - → Reduces glitches and errors in sequential digital circuits.
- **Applications in Karnaugh Maps (K-Maps)**
  - →Gray Code is used for cell numbering in K-maps so that adjacent cells differ by only one bit
  - → simplifying logic minimization.

Gray Code

Binary to Gray Code Conversion

- Keep the MSB the same.
- Each next Gray bit = (previous Binary bit) XOR (current Binary bit).

XOR logic:
$0 \oplus 0 = 0$, $1 \oplus 1 = 0$,
$1 \oplus 0 = 1$, $0 \oplus 1 = 1$.

Example: Convert 1011_2 to Gray Code.

MSB = 1
$1 \oplus 0 = 1$
$0 \oplus 1 = 1$
$1 \oplus 1 = 0$

Gray Code: 1110

Gray to Binary Code Conversion

- Keep the MSB the same.
- Each next Binary bit = (next Binary bit) XOR (Gray bit).

Example: Convert 1110 to Binary Code.

MSB = 1
$1 \oplus 1 = 0$
$0 \oplus 1 = 1$
$1 \oplus 0 = 1$

Binary Code: 1011_2

Alphanumeric Code

- Alphanumeric codes represent letters (A–Z), digits (0–9), and symbols in binary form for digital systems.
- Used for data storage, communication, and processing in computers.

Common Types

1.ASCII (American Standard Code for Information Interchange)

- 7-bit (128 chars), later 8-bit extended (256 chars).
- Example: A = 65 (01000001)

2.EBCDIC (Extended Binary Coded Decimal Interchange Code)

- 8-bit, developed by IBM for mainframes.

3.UNICODE

- 16-bit or more, represents all world languages & symbols.
- Example: Supports Hindi, Chinese, Emojis, etc.

ASCII Code

American Standard Code for Information Interchange (ASCII)

	$b_7b_6b_5$							
$b_4b_3b_2b_1$	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	|
1101	CR	GS	–	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Control characters

NUL	Null	DLE	Data-link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell	ETB	End-of-transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete

Error Detection and Correction Codes

Parity Code for Error Detection:

The parity of the given binary word is defined by whether the number of 1's is *odd or even*.

Even Parity: Adding a parity bit makes the number of 1's in a word *even*.

Odd Parity: Adding a parity bit makes the number of 1's in a word *odd*.

Decimal	8421 BCD	Even Parity	Odd Parity
0	0 0 0 0	0	1
1	0 0 0 1	1	0
2	0 0 1 0	1	0
3	0 0 1 1	0	1
4	0 1 0 0	1	0
5	0 1 0 1	0	1
6	0 1 1 0	0	1
7	0 1 1 1	1	0
8	1 0 0 0	1	0
9	1 0 0 1	0	1

Error Detection and Correction Codes

Hamming Code for Error Correction:

To transmit the four-bit data, three parity bits are located at positions (1) 2^0, (2) 2^1, and (4) 2^2,

$$P_1\ P_2\ D_3\ P_4\ D_5\ D_6\ D_7$$

P_1 is set to be 0 or 1, so that it establishes even parity over bits 1, 3, 5, and 7.

P_2 is set to be 0 or 1, so that it establishes even parity over bits 2, 3, 6, and 7.

P_3 is set to be 0 or 1, so that it establishes even parity over bits 4, 5, 6, and 7.

Check bits:

$C_1 = P_1 \oplus D_3 \oplus D_5 \oplus D_7$

$C_2 = P_1 \oplus D_3 \oplus D_6 \oplus D_7$

$C_3 = D_4 \oplus D_5 \oplus D_6 \oplus D_7$

If $C_3\ C_2\ C_1 = 0\ 0\ 0$, there is no error

Else, the error bit position is given by $C_{3n}C_{2n}C_1$

Error Detection and Correction Codes

Hamming Code for Error Correction:

Example: The code sent is 1001100, and the code received is 1001110? Find the error position.

Check bits:

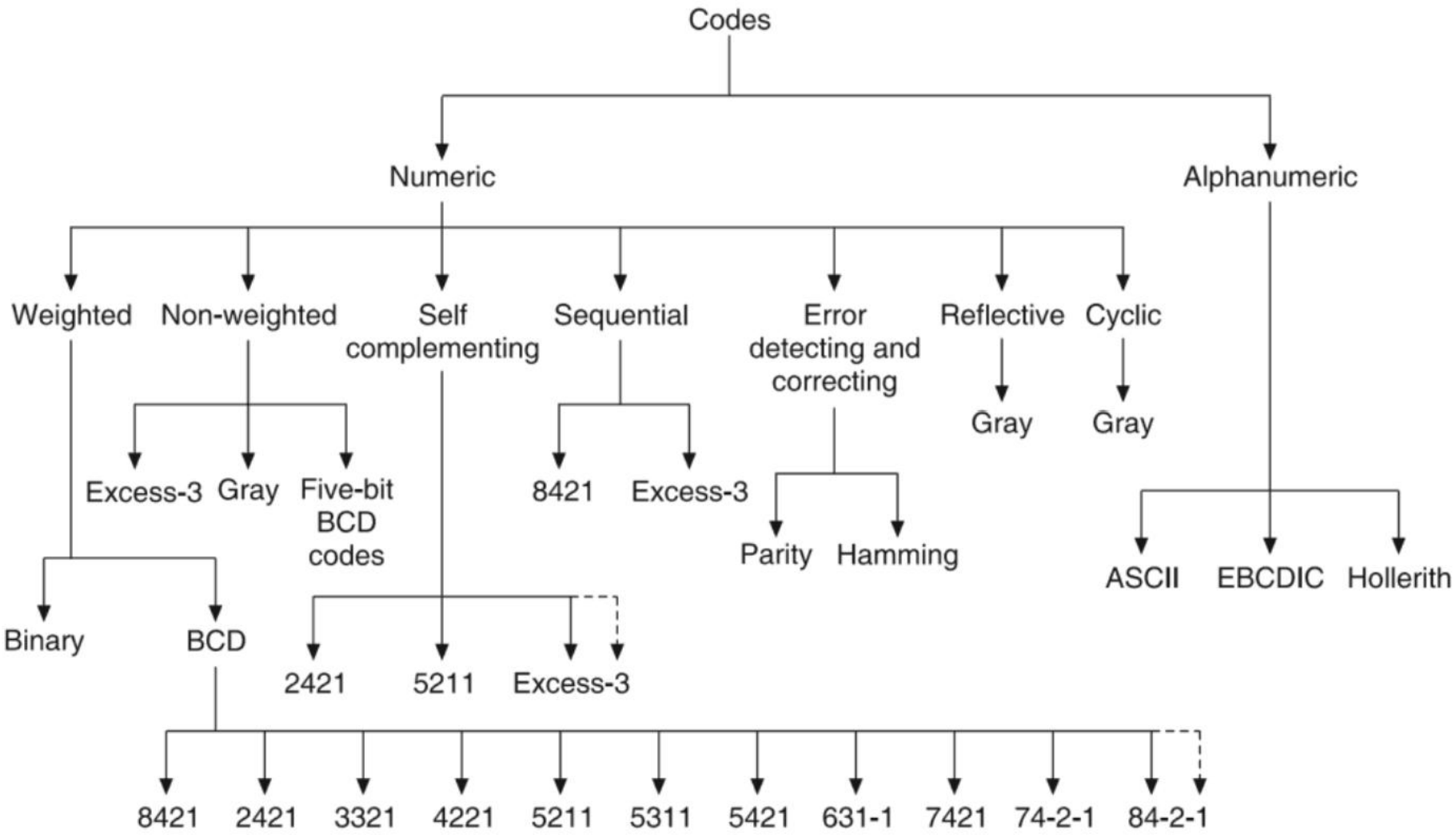
$C_1 = P_1 \oplus D_3 \oplus D_5 \oplus D_7 = 0$

$C_2 = P_2 \oplus D_3 \oplus D_6 \oplus D_7 = 1$

$C_3 = P_4 \oplus D_5 \oplus D_6 \oplus D_7 = 1$

Error position is $C_3\ C_2\ C_1 = 1\ 1\ 0$ (6-bit)

Overall Classification of Binary Codes



Logical Gates

- ***Logic gates are fundamental building blocks of digital systems.***
- ***Integrated circuits or chips contain a large number of logic gates.***

Three basic operations:

- ➔ OR operation
- ➔ AND operation
- ➔ NOT operation

VLSI technology uses 10^4 gates/chips

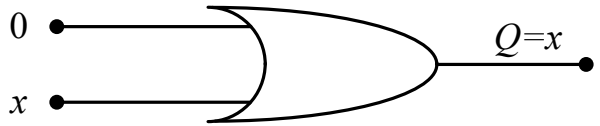
Enable input: The input that allows other inputs to pass through the gate.

Disable input: The input that does not allow other inputs to pass through the gate.

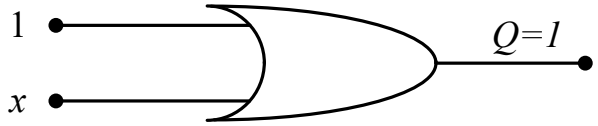
Basic Gates:

OR Gate

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1



In the OR gate, logic “0” is an enabled input.



In the OR gate, logic “1” is a disabled input.

Basic Logic Gates

Examples:

WiFi OR Mobile Data → Internet 📶 works (if either is ON, reels play)

Face ID OR Passcode → You can unlock your phone 📱.

Login with Google OR Facebook → You can access the app 📲.

OR Gate = at least one option works.
It's the "backup plan" logic Gen Z always uses 😉.

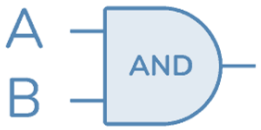
Three-input OR Gate:

Q = A+B+C

Input A	Input B	Input C	Output Q
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

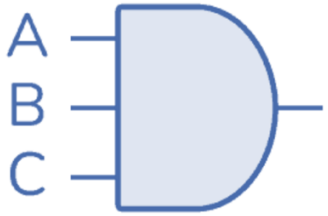
Basic Logic Gates

AND Gate:



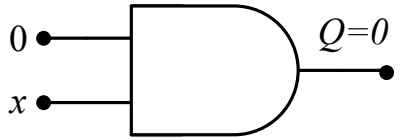
$Q = A \cdot B$

A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

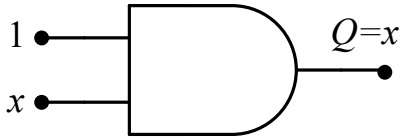


$Q = A \cdot B \cdot C$

Input A	Input B	Input C	Output Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1



logic “0” is a disabled input.



logic “1” is an enabled input.

Basic Logic Gates

Examples:

Email AND Password
→ Instagram login 🔋 .

Camera Permission AND Storage Permission
→ Instagram camera works. 🤳

Follow AND Accept Request
→ You become Instagram friends. 🤝

Attendance AND Marks
→ To Pass the "**Digital Electronics**" course 😎

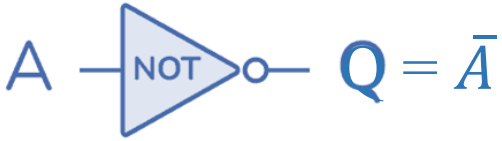
Guess the Logic Gate:

Marriage Logic Gate

Man	Woman	Result
Wrong	Right	Woman is Right
Right	Right	Woman is Right
Right	Wrong	Woman is Right
Wrong	Wrong	Man is Wrong

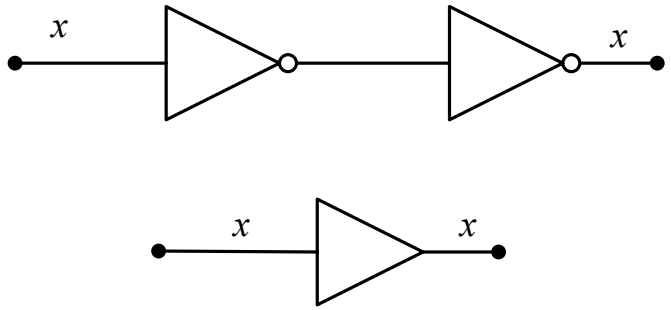
Basic Logic Gates

NOT Gate (Inverter):



A	Q
0	1
1	0

Buffer:



Examples:

Dark Mode Toggle → If ON → screen goes dark; if OFF → screen is bright.

Mute Button → If input = sound ON → output = silent; if input = sound OFF → output = loud.

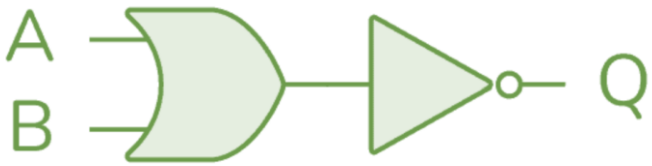
Airplane Mode → If ON → internet OFF; if OFF → internet ON.

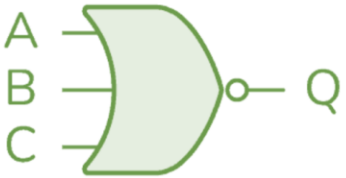
Instagram Private Account → If ON → strangers can't follow; if OFF → everyone can

Universal Logic Gates

NOR Gate:

NOR = OR + NOT ➔

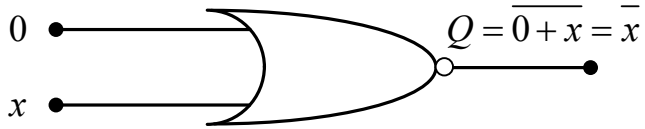




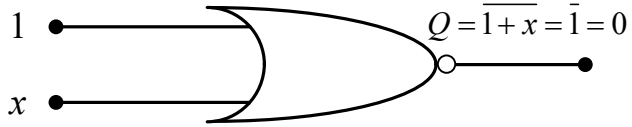
Input A	Input B	Input C	Output Q
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

$$Q = \overline{A + B}$$

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0



logic “0” is an enabled input.



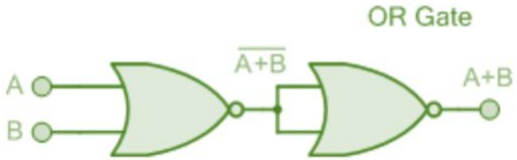
logic “1” is a disabled input.

Universal Logic Gates

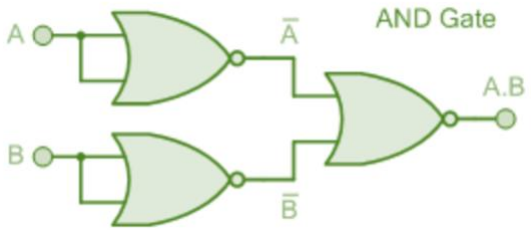
NOR Gate is a Universal logic gate:

NOT Operation:

OR Operation:



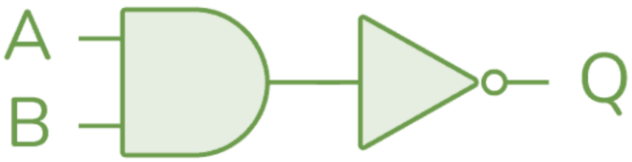
AND Operation:

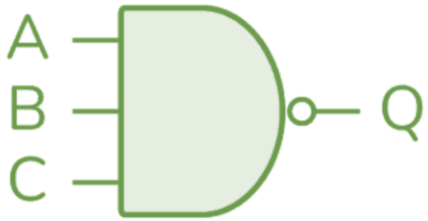


Universal Logic Gates

NAND Gate:

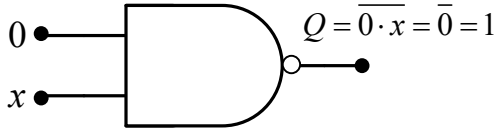
NAND = AND + NOT ➔



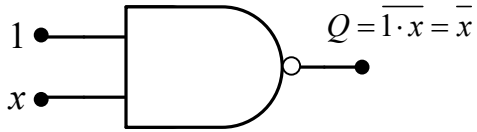


A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0

Input A	Input B	Input C	Output Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0



logic “0” is a disabled input.



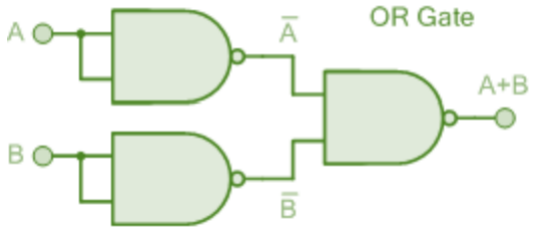
logic “1” is an enabled input.

Universal Logic Gates

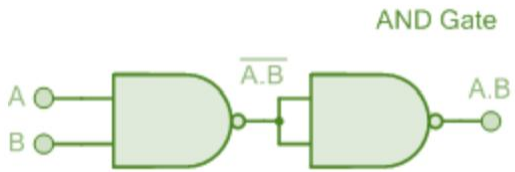
NAND Gate is a Universal logic gate:

NOT Operation:

OR Operation:



AND Operation:

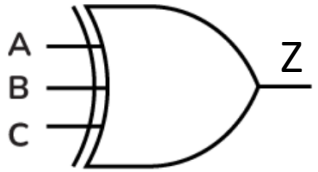


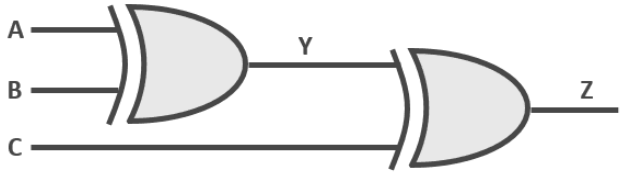
Special Logic Gates

XOR or Ex-OR Gate:

$$Q = A \oplus B = A\bar{B} + \bar{A}B$$

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0





A	B	C	Y	Z
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	0	1

Examples:

Login Options

→ You can log in with Google **or** Facebook, but not both at the same time.

Earphones 🎧

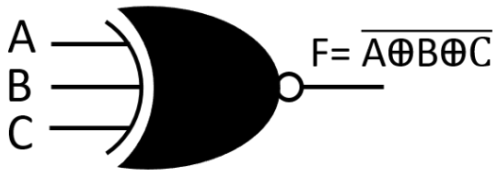
→ You can connect either Bluetooth **or** Wired headphones — both won't work together.

Special Logic Gates

XNOR or Ex-NOR Gate:

$$Q = \overline{A \oplus B} = \bar{A}\bar{B} + AB$$

A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1



A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Examples:

Face ID Unlock

→ Phone unlocks only if **your face matches the stored face**.

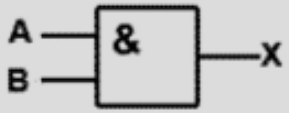
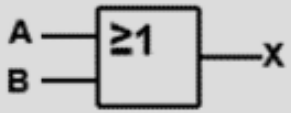
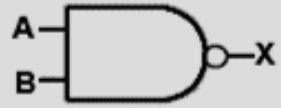
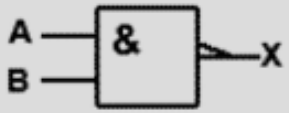
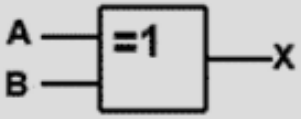
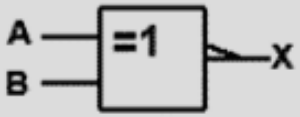
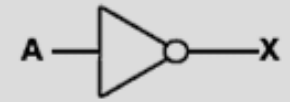
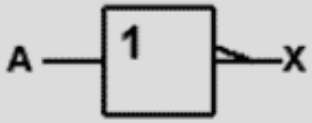
Password Check

→ Login works when **entered password = saved password**.

Bluetooth Pairing

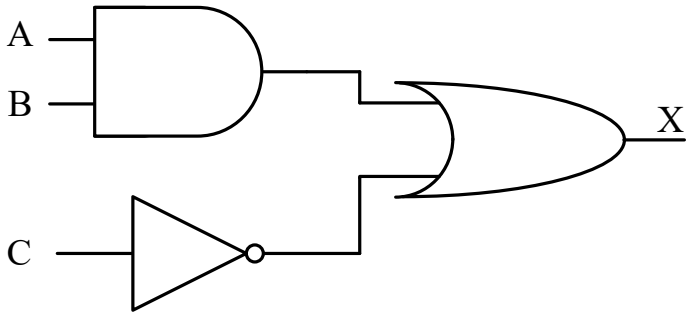
→ Device connects only if **PIN entered = PIN expected**.

Summary of Logic Gates

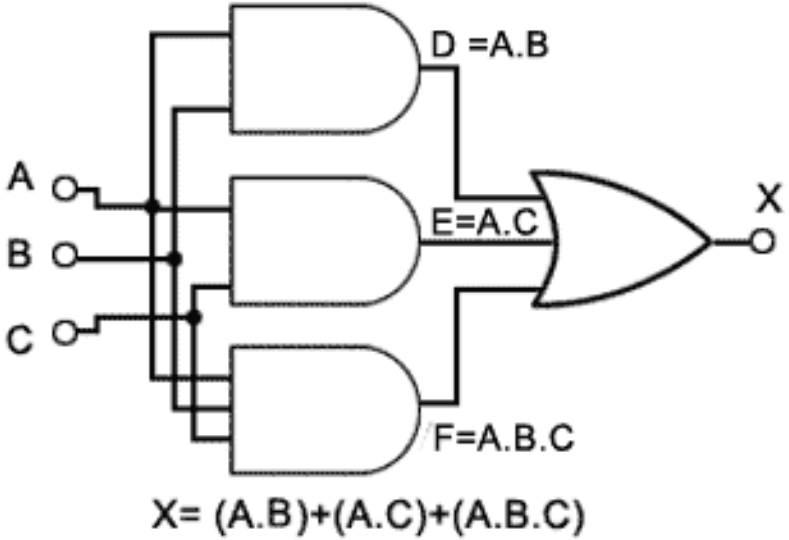
ANSI Symbol	IEC Symbol	Description	Boolean
		The AND gate output is at logic 1 when, and only when all its inputs are at logic 1, otherwise the output is at logic 0.	$X = A \cdot B$
		The OR gate output is at logic 1 when one or more of its inputs are at logic 1. If all the inputs are at logic 0, the output is at logic 0.	$X = A+B$
		The NAND Gate output is at logic 0 when, and only when all its inputs are at logic 1, otherwise the output is at logic 1.	$X = \overline{A \cdot B}$
		The NOR gate output is at logic 0 when one or more of its inputs are at logic 1. If all the inputs are at logic 0, the output is at logic 1.	$X = \overline{A+B}$
		The XOR gate output is at logic 1 when one and ONLY ONE of its inputs is at logic 1. Otherwise the output is logic 0.	$X = A \oplus B$
		The XNOR gate output is at logic 0 when one and ONLY ONE of its inputs is at logic 1. Otherwise the output is logic 1. (It is similar to the XOR gate, but its output is inverted).	$X = \overline{A \oplus B}$
		The NOT gate output is at logic 0 when its only input is at logic 1, and at logic 1 when its only input is at logic 0. For this reason it is often called an INVERTER.	$X = \overline{A}$

Logic Gate Representation

X = (A•B) + C'



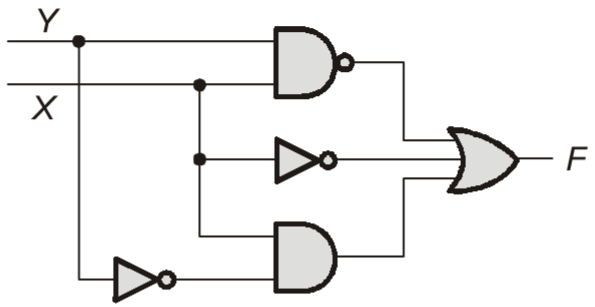
X = (A•B) + (A•C) + (A•B•C)



Derive the logic diagram with two-input logic gates.

Examples on Logic Gates:

Q.31 Consider the following logic circuit diagram.



Which is/are the CORRECT option(s) for the output function?

(a) $\overline{XY}$

(b) $\overline{X}+\overline{Y}+X\overline{Y}$

(c) $\overline{XY}+\overline{X}+X\overline{Y}$

(d) $X+\overline{Y}$

[2025 (Set-2) : 1 M]

Answer: (a, b, c)

Boolean Algebra

❑ Boolean or Switching Algebra:

→ An algebraic system defined on the set {0,1}, with two binary operations {AND, OR} and one unary operation (NOT)

→ AND operation is a logical product, which is denoted with ‘ · ’ or ‘Λ’

-- **Ex:** 1·0 = 0·1= 0·0 = 0, 1·1 =1

→ OR operation is a logical addition, which is denoted with ‘+’ or ‘V’

-- **Ex:** 1+0 = 0+1 = 1+1 = 1, and 0+0 = 0.

→ NOT operation is a logical complement, which is denoted by a single quote (*A’*) or $\bar{A}$

-- **Ex:** $\bar{0} = 1$, and $\bar{1} = 0$

❑ Switching Variables:

→ two-valued variables that can take on two distinct values, 0 and 1.

-- **Ex:** ‘*A’* is a switching variable which can take ‘0’ or ‘1’)

❑ Switching Expression:

→ An expression consists of switching variables, constants, and operators.

-- **Ex:** $A \cdot B + A \cdot C'$

Basic Laws of Boolean Algebra

Let literals *x, y, z* be switching variables …..

- Basic Identities:

$$x + 1 = 1$$
$$x + 0 = x$$
$$x \cdot 1 = x$$
$$x \cdot 0 = 0$$

- Idempotent Law:

$$x + x = x$$
$$x \cdot x = x$$

- Commutative Law:

$$x + y = y + x$$
$$x \cdot y = y \cdot x$$

- Complementation Law:

$$x + \bar{x} = 1$$
$$x \cdot \bar{x} = 0$$

- Associative Law:

$$(x + y) + z = x + (y + z)$$
$$(x \cdot y) \cdot z = x \cdot (y \cdot z)$$

- Distributive Law:

$$x \cdot (y + z) = x \cdot y + x \cdot z$$
$$x + (y \cdot z) = (x + y) \cdot (x + z)$$

- Absorption Law:

$$x + (x \cdot y) = x$$
$$x \cdot (x + y) = x$$

- Useful Law:

$$x + (\bar{x} \cdot y) = x + y$$
$$x \cdot (\bar{x} + y) = x \cdot y$$

- De Morgan's Theorem:

$$\overline{(x + y)} = \bar{x} \cdot \bar{y}$$
$$\overline{(x \cdot y)} = \bar{x} + \bar{y}$$

Basic Laws of Boolean Algebra

- Consensus Theorem:

$$x \cdot y + \bar{x} \cdot z + y \cdot z = x \cdot y + \bar{x} \cdot z$$
$$(x + y) \cdot (\bar{x} + z) \cdot (y + z) = (x + y) \cdot (\bar{x} + z)$$

- Involution:

$$\bar{\bar{x}} = x$$

- Duality: A Boolean expression X_2 can be obtained from a given Boolean expression X_1 by interchanging the operations AND & OR, and constants 0 &1. Then, X_1 and X_2 are dual to each other.

  Ex: $x + 1 = 1$ *is the dual of* $x \cdot 0 = 0$

  $x + x \cdot y = x$ *is the dual of* $x \cdot (x + y) = x$

  $x + \bar{x} \cdot y = x + y$ *is the dual of* $x \cdot (\bar{x} + y) = x \cdot y$

- Note:
  - → *x+y = x+z does not imply y=z*
  - → *Inverse operations are not defined in Boolean algebra; hence, cancellations are not allowed.*
  - → Ex*: x=1, y=0, and z=1*

Simplification of Boolean Expression

- Draw the logic diagram of the expression below, and simplify it by using the algebra rules:

$$X = A\bar{B} + AB + BC$$

- Draw the 2-input logic diagram of the expression below, and simplify it by using the algebra rules:

$$X = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

Simplification of Boolean Expression

- Simplify the expression below:

$$X = A\left[B + \bar{C}\left(\overline{AB + A\bar{C}}\right)\right]$$

Answer: AB

Examples on Simplification of Boolean Expression

- Gate2025 (1M): Let X be the 3-variable Boolean expression that produces the output as '1' when at least two of the input variables are '1'. What is the expression of X?

Answer: X(A,B,C) = AB+BC+CA

- Gate2024 (2M): **Q.55** Consider the operators ◊ and □ defined by $a \lozenge b = a + 2b$, $a \square b = ab$, for positive integers. Which of the following statements is/are TRUE?
  (a) Operator ◊ obeys the associative law.
  (b) Operator ◊ over the operator □ obeys the distributive law.
  (c) Operator □ obeys the associative law.
  (d) Operator □ over the operator ◊ obeys the distributive law.

Answer: (c, d)

Examples on Simplification of Boolean Expression

Q.50 Which of the following Boolean algebraic equation(s) is/are CORRECT?

(a) $\bar{A}BC + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} + A\bar{B}C + ABC = BC + \bar{B}\bar{C} + \bar{A}\bar{B}$

(b) $AB + \bar{A}C + BC = AB + \bar{A}C$

(c) $(A+C)(\bar{A}+B) = AB + \bar{A}C$

(d) $\overline{(A+\bar{B}+\bar{D})(C+D)(\bar{A}+C+D)(A+B+\bar{D})} = \bar{A}D + \bar{C}\bar{D}$

[2025 (Set-2) : 2 M]

Answer: (b, c, d)

Simplification of Boolean Expression

- In SRM AP, safe locks of hostels Godavari, Krishna, Narmada, Yamuna, Kaveri, all of which must be unlocked to open. The keys are distributed among five security officers as below.
  - → Officer A has the keys for the locks Godavari and Narmada
  - → Officer B has the keys for the locks Godavari and Yamuna
  - → Officer C has the keys for the locks Krishna and Yamuna
  - → Officer D has the keys for the locks Narmada and Kaveri
  - → Officer E has the keys for the locks Godavari and Kaveri

Find all combinations of security officers that can open the safe.

$$X = Godavari \cdot Krishna \cdot Narmada \cdot Yamuna \cdot Kaveri$$

$$X = (A + B + E) \cdot (C) \cdot (A + D) \cdot (B + C) \cdot (D + E)$$

Answer: ACE+ACD+BCD+CDE

Boolean Expressions and Their Representations

- **Literal:** For a Boolean expression, a ***literal*** is defined as a variable in uncomplemented or complemented form.
  - Example: *x, x', y, y'*
  - *A'B'C+DE* → 5 literals

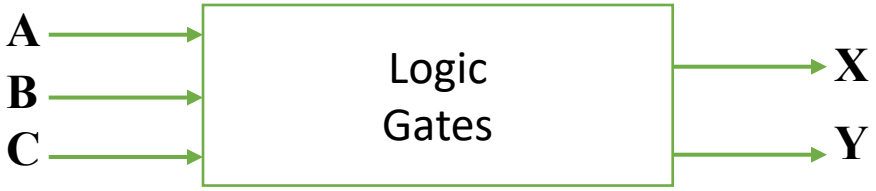
Considering an n-variable Boolean expression $f(x_1, x_2, \ldots, x_n)$.

- **Minterm:** A product term (that is, an AND operation) of all the n-literals is called a ***minterm***.
  - Example: $f(A, B, C) = (A \cdot \bar{B} \cdot C), (\bar{A} \cdot B.\bar{C})$
  - A minterm assumes the value 1 for exactly one combination of variables.
    - ✓ Ex: $(A \cdot \bar{B} \cdot C)$ ➔ (A=1, B=0, C=1), $(\bar{A} \cdot B.\bar{C})$ ➔ (A=0, B=1, C=0)

- **Maxterm:** A sum term (that is, an OR operation) of all the n-literals is called a ***maxterm***.
  - Example: $f(A, B, C) = (A + \bar{B} + C), (\bar{A} \cdot B.\bar{C})$
  - A maxterm assumes the value 0 for exactly one combination of variables.
    - ✓ Ex: $(A + \bar{B} + C)$ ➔ (A=0, B=1, C=0), $(\bar{A} + B + \bar{C})$ ➔ (A=1, B=0, C=1)

Canonical Forms of Boolean Expressions

- **Sum of Product (SOP) Form:** Take the sum of minterms.



A	B	C	X	Y
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$X = (\bar{A} \cdot \bar{B} \cdot C) + (\bar{A} \cdot B \cdot \bar{C}) + (A \cdot \bar{B} \cdot \bar{C}) + (A \cdot B \cdot C)$$

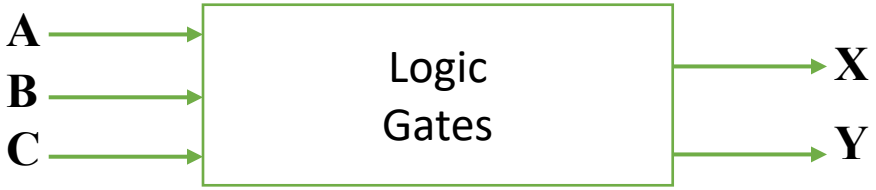
$$X = m_1 + m_2 + m_4 + m_7 \rightarrow X = \sum(1,2,4,7)$$

$$Y = (A \cdot B \cdot \bar{C}) + (\bar{A} \cdot B \cdot C) + (A \cdot \bar{B} \cdot C) + (A \cdot B \cdot C)$$

$$Y = m_3 + m_5 + m_6 + m_7 \rightarrow \mathrm{Y} = \sum(3,5,6,7)$$

Canonical Forms of Boolean Expressions

- **Product of Sums (POS) Form:** Take the product of maxterms.



A	B	C	X	Y
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$X = (A + B + C) \cdot (A + \bar{B} + \bar{C}) \cdot (\bar{A} + B + \bar{C}) \cdot (\bar{A} + \bar{B} + C)$$

$$X = M_0 + M_3 + M_5 + M_6 \rightarrow X = \prod(0,3,5,6)$$

$$Y = (A + B + C) + (A + B + \bar{C}) + (A + \bar{B} + C) + (\bar{A} + B + C)$$

$$Y = M_0 + M_1 + M_2 + M_4 \rightarrow \mathrm{Y} = \prod(0,1,2,4)$$

Canonical Forms of Boolean Expressions

Algebraic procedure to obtain Sum of Product (SOP) Form:

- Examine each term of a given SOP expression; if it is not a minterm, go to the next step.
- For all missing variable x_i, multiply the term by $(x_i + \overline{x_i})$
- Multiply out all the products and eliminate the redundant product terms.

Example: $f(A,B,C) = A \cdot \bar{B} + B + A \cdot B \cdot C$

$$= A \cdot \bar{B} \cdot (C + \bar{C}) + B \cdot (A + \bar{A}) \cdot (C + \bar{C}) + (A \cdot B \cdot C)$$

$$= A \cdot \bar{B} \cdot C + A \cdot \bar{B} \cdot \bar{C} + A \cdot B \cdot C + A \cdot B \cdot \bar{C} + \bar{A} \cdot B \cdot C + \bar{A} \cdot B \cdot \bar{C} + A \cdot B \cdot C$$

$$= A \cdot \bar{B} \cdot C + A \cdot \bar{B} \cdot \bar{C} + A \cdot B \cdot C + A \cdot B \cdot \bar{C} + \bar{A} \cdot B \cdot C + \bar{A} \cdot B \cdot \bar{C}$$

$$= m_2 + m_3 + m_4 + m_5 + m_6 + m_7 = \sum m(2,3,4,5,6,7)$$

Canonical Forms of Boolean Expressions

Algebraic procedure to obtain Product of Sums (POS) Form:

- Examine each term of a given POS expression; if it is not a maxterm, go to the next step.
- For all missing variable x_i, sum the term with $(x_i \cdot \overline{x_i})$
- Obtain the sum terms, and eliminate the redundant sum terms.

Example: $f(A,B,C) = \bar{A} \cdot (\bar{B} + C)$

$$= (\bar{A} + B \cdot \bar{B} + C \cdot \bar{C}) + (\bar{B} + C + A \cdot \bar{A})$$

$$= (\bar{A} + B + C) \cdot (\bar{A} + B + \bar{C}) \cdot (\bar{A} + \bar{B} + C) \cdot (\bar{A} + \bar{B} + \bar{C}) \cdot (A + \bar{B} + C)$$

$$= M_2 + M_4 + M_5 + M_6 + M_7 = \prod(2,4,5,6,7)$$

Canonical Forms of Boolean Expressions

❑ Conversion of one Form to another Form:

- Double complement the given function and apply De Morgan's theorem.

Example: $f(A,B,C) = \sum m(0,2,4,6,7)$

Apply first complement → $\overline{f(A,B,C)} = \sum m(1,3,5)$ (write the missing minterms)

Apply second complement → $f = \overline{\sum m(1,3,5)} = \overline{m_1} \cdot \overline{m_3} \cdot \overline{m_5}$ (use De Morgans's theorem)

$$\overline{m_1} \cdot \overline{m_3} \cdot \overline{m_5} = M_1\ M_3\ M_5 = \prod(1,3,5)$$

Canonical Forms of Boolean Expressions

- **Expand $\bar{A} + \bar{B}$ into minterms:**

Example: $f(A,B) = \bar{A} \cdot (B + \bar{B}) + (A + \bar{A}) \cdot \bar{B}$

$$= \bar{A} \cdot B + \bar{A} \cdot \bar{B} + A \cdot \bar{B} + \bar{A} \cdot \bar{B}$$
$$= \bar{A} \cdot B + \bar{A} \cdot \bar{B} + A \cdot \bar{B}$$
$$= 01 + 00 + 01$$
$$= m_0 + m_1 + m_2 = \sum m(0,1,2)$$

Shortcut: $f(A,B) = \bar{A} \cdot X + X \cdot \bar{B}$

$$= 0 \cdot X + X \cdot 0$$
$$= 00 + 01 + 00 + 10$$
$$= 00 + 01 + 10$$
$$= m_0 + m_1 + m_2 = \sum m(0,1,2)$$

Canonical Forms of Boolean Expressions

- **Expand $A \cdot (\bar{A} + B) \cdot (\bar{A} + B + \bar{C})$ into maxterms and minterms:**

Example: $f(A,B,C) = A \cdot (\bar{A} + B) \cdot (\bar{A} + B + \bar{C})$

$$\mathrm{A} \rightarrow A + B \cdot \bar{B} + C \cdot \bar{C} = (A + B + C) \cdot (A + B + \bar{C}) \cdot (A + \bar{B} + C) \cdot (A + \bar{B} + \bar{C}) \cdot$$

$$\bar{A} + B \rightarrow \bar{A} + B + C \cdot \bar{C} = (\bar{A} + B + C) \cdot (\bar{A} + B + \bar{C})$$

$$A \cdot (\bar{A} + B) \cdot (\bar{A} + B + \bar{C}) = (000) \cdot (001) \cdot (010) \cdot (011) \cdot (100) \cdot (101)$$

$$= M_0 \cdot M_1 \cdot M_2 \cdot M_3 \cdot M_4 \cdot M_5 = \prod M(0,1,2,3,4,5)$$

Shortcut: $f(A,B,C) = A \cdot (\bar{A} + B) \cdot (\bar{A} + B + \bar{C})$

$$\mathrm{A} \rightarrow 0XX \rightarrow (000),(001),(010),(011) = M_0 \cdot M_1 \cdot M_2 \cdot M_3$$

$$\bar{A} + B \rightarrow 10X \rightarrow (100),(101) = M_4 \cdot M_5$$

$$\bar{A} + B + \bar{C} \rightarrow (101) = M_5$$

$$f(A,B,C) = M_0 \cdot M_1 \cdot M_2 \cdot M_3 \cdot M_4 \cdot M_5$$

Canonical Forms of Boolean Expressions

- **Expand $A \cdot (\bar{A} + B) \cdot (\bar{A} + B + \bar{C})$ into maxterms and minterms:**

Example: $f(A, B, C) = A \cdot (\bar{A} + B) \cdot (\bar{A} + B + \bar{C})$

$$\mathrm{A} \rightarrow A + B \cdot \bar{B} + C \cdot \bar{C} = (A + B + C) \cdot (A + B + \bar{C}) \cdot (A + \bar{B} + C) \cdot (A + \bar{B} + \bar{C}) \cdot$$

$$\bar{A} + B \rightarrow \bar{A} + B + C \cdot \bar{C} = (\bar{A} + B + C) \cdot (\bar{A} + B + \bar{C})$$

$$A \cdot (\bar{A} + B) \cdot (\bar{A} + B + \bar{C}) = (000) \cdot (001) \cdot (010) \cdot (011) \cdot (100) \cdot (101)$$

$$= M_0 \cdot M_1 \cdot M_2 \cdot M_3 \cdot M_4 \cdot M_5 = \prod M(0,1,2,3,4,5)$$

Minterms: $f(A, B, C) = \prod M(0,1,2,3,4,5)$

Apply first complement → $\overline{f(A, B, C)} = \prod M(6,7)$(write the missing minterms)

Apply second complement → $f = \overline{\prod M(6,7)} = \overline{M_6} + \overline{M_7}$ (use De Morgans's theorem)

$$\overline{M_6} + \overline{M_7} = m_6 + m_7 = (A \cdot \bar{B} \cdot C) + (A \cdot B \cdot C)$$

Canonical Forms of Boolean Expressions

- **Expand $A + B\bar{C} + AB\bar{D} + ABCD$ into maxterms and minterms:**

Example: $f(A,B,C,D) = A + B\bar{C} + AB\bar{D} + ABCD$

$$A \rightarrow A(B+\bar{B})(C+\bar{C})(D+\bar{D})$$

$$B\bar{C} \rightarrow B\bar{C}(A+\bar{A})(D+\bar{D})$$

$$AB\bar{D} \rightarrow AB\bar{D}(C+\bar{C})$$

$$A + B\bar{C} + AB\bar{D} + ABCD = \sum m(4,5,8,9,10,11,12,13,14,15)$$

So, Maxterms: $\prod M(0,1,2,3,6,7)$

Shortcut: $f(A,B,C,D) = A + B\bar{C} + AB\bar{D} + ABCD$

$$A \rightarrow AXXX \rightarrow 1XXX \rightarrow 1000 + 1001 + 1010 + 1011 + 1100 + 1101 + 1110 + 1111$$

$$B\bar{C} \rightarrow XB\bar{C}X \rightarrow X10X \rightarrow 0100+0101+1100+1101$$

$$AB\bar{D} \rightarrow ABX\bar{D} \rightarrow 11X0 \rightarrow 1100 + 1110$$

$$ABCD \rightarrow 1111$$

$$= \sum m(4,5,8,9,10,11,12,13,14,15)$$

Minimization using Karnaugh Maps

- **Karnaugh Maps (K-Map):** It is graphical method of minimizing Boolean expressions, without using algebraic theorems.

- **K-Map Method:**
  - Fill up the cells of the K-map with minterms of the function
  - Group the minterms into maximum cubes, and groups can only contain 1, 2, 4, 8, 16 or 32... etc. cells (powers of 2), and every minterm should be covered with one cube.
  - Write down the SOP expression by creating one product term out of each cube.
  - For an **n**-variable expression, there are 2^n cells in the map (one for each minterm)

    → Adjacent *2* cells differ in *1* variable

    → Adjacent $2^2=4$ cells differ in *2* variables

    → Adjacent 2^m cells differ in *m* variable

- **Drawbacks of K-Maps:** **It is difficult visualize expressions with more than 5 variables.**

Minimization using Karnaugh Maps

Two-variable K-Map:

- Two input variables: A, B
- Four output combinations (minterms): $\bar{A}\bar{B}, \bar{A}B, A\bar{B}, AB$

A \ B		
	$\bar{A}\bar{B}$	$\bar{A}B$
	$A\bar{B}$	AB

A \ B	$\bar{B}$	B
$\bar{A}$	00	01
A	10	11

A \ B	0	1
0	0	1
1	2	3

Simplify $\boldsymbol{f(A,B) = \sum m(0,1), \sum m(0,2), \sum m(1,3), \sum m(2,3), \sum m(0,1,2,3)}$

A \ B	$\bar{B}$	B
$\bar{A}$	1 (0)	1 (1)
A	0 (2)	0 (3)

$f = \bar{A}$

A \ B	$\bar{B}$	B
$\bar{A}$	1 (0)	0 (1)
A	1 (2)	0 (3)

$f = \bar{B}$

A \ B	$\bar{B}$	B
$\bar{A}$	0 (0)	1 (1)
A	0 (2)	1 (3)

$f = B$

A \ B	$\bar{B}$	B
$\bar{A}$	0 (0)	0 (1)
A	1 (2)	1 (3)

$f = A$

A \ B	$\bar{B}$	B
$\bar{A}$	1 (0)	1 (1)
A	1 (2)	1 (3)

$f = 1$

Minimization using Karnaugh Maps

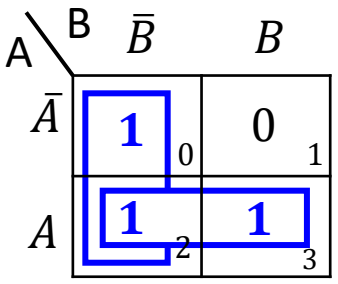
Two-variable K-Map:

- Two input variables: A, B
- Four output combinations (minterms): $\bar{A}\bar{B}, \bar{A}B, A\bar{B}, AB$

Simplify $f(A,B) = \sum m(0,2,3)$ using Boolean algebra and K-map?

$$f(A,B) = \sum m(0,2,3)$$

$$= m_0 + m_1 + m_3$$
$$= \bar{A}\bar{B} + A\bar{B} + AB$$
$$= \bar{B}(\bar{A} + A) + AB$$
$$= \bar{B} + A \cdot B$$
$$= (\bar{B} + A) \cdot (\bar{B} + B)$$
$$= (\bar{B} + A)$$



$$f = A + \bar{B}$$

Minimization using Karnaugh Maps

Three-variable K-Map:

- Two input variables: A, B, C
- Eight output combinations (minterms): $\bar{A}\bar{B}\bar{C}, \bar{A}\bar{B}C, \bar{A}B\bar{C}, \bar{A}BC, A\bar{B}\bar{C}, A\bar{B}C, AB\bar{C}, ABC$

A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110

A \ BC	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	$\bar{A}BC$	$\bar{A}B\bar{C}$
A	$A\bar{B}\bar{C}$	$A\bar{B}C$	ABC	$AB\bar{C}$

A \ BC	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	0	1	3	2
A	4	5	7	6

Minimization using Karnaugh Maps

$f = \sum(0,4) = \overline{B}\,\overline{C}$

A \ BC	00	01	11	10
0	1	0	0	0
1	1	0	0	0

$f = \sum(4,5) = A\,\overline{B}$

A \ BC	00	01	11	10
0	0	0	0	0
1	1	1	0	0

$f = \sum(0,1,4,5) = \overline{B}$

A \ BC	00	01	11	10
0	1	1	0	0
1	1	1	0	0

$f = \sum(0,1,2,3) = \overline{A}$

A \ BC	00	01	11	10
0	1	1	1	1
1	0	0	0	0

$f = \sum(0,4) = \overline{A}\,C$

A \ BC	00	01	11	10
0	0	1	1	0
1	0	0	0	0

$f = \sum(4,6) = A\,\overline{C}$

A \ BC	00	01	11	10
0	0	0	0	0
1	1	0	0	1

$f = \sum(0,2) = \overline{A}\,\overline{C}$

A \ BC	00	01	11	10
0	1	0	0	1
1	0	0	0	0

$f = \sum(0,2,4,6) = \overline{C}$

A \ BC	00	01	11	10
0	1	0	0	1
1	1	0	0	1

Minimization using Karnaugh Maps

A \ BC	00	01	11	10
0		1		
1	1		1	1

$$f = AB + A\bar{B}\bar{C} + \bar{A}\bar{B}C$$

A \ BC	00	01	11	10
0	1		1	1
1	1			1

$$f = \bar{C} + \bar{A}B$$

A \ BC	00	01	11	10
0			1	1
1	1	1		

$$f = A\bar{B} + \bar{A}B$$

A \ BC	00	01	11	10
0			1	
1	1		1	1

$$f = BC + A\bar{C}$$

A \ BC	00	01	11	10
0		1	1	1
1		1	1	

$$f = C + \bar{A}B$$

A \ BC	00	01	11	10
0	1		1	1
1	1		1	1

$$f = \bar{C} + B$$

Ramanujan Magic Square

10	11	19	94
95	18	8	13
9	12	96	17
20	93	11	10

10	11	19	94
95	18	8	13
9	12	96	17
20	93	11	10

10	11	19	94
95	18	8	13
9	12	96	17
20	93	11	10

10	11	19	94
95	18	8	13
9	12	96	17
20	93	11	10

10	11	19	94
95	18	8	13
9	12	96	17
20	93	11	10

10	11	19	94
95	18	8	13
9	12	96	17
20	93	11	10

10	11	19	94
95	18	8	13
9	12	96	17
20	93	11	10

10	11	19	94
95	18	8	13
9	12	96	17
20	93	11	10

Minimization using Karnaugh Maps

Four-variable K-Map:

- Two input variables: A, B, C, D
- Sixteen output combinations (minterms):

AB \ CD	00	01	11	10
00	0000	0001	0011	0010
01	0100	0101	0111	0110
11	1100	1101	1111	1110
10	1000	1001	1011	1010

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	1	3	2
$\bar{A}B$	4	5	7	6
AB	12	13	15	14
$A\bar{B}$	8	9	11	10

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	$\bar{A}\bar{B}\bar{C}\bar{D}$	$\bar{A}\bar{B}\bar{C}D$	$\bar{A}\bar{B}CD$	$\bar{A}\bar{B}C\bar{D}$
$\bar{A}B$	$\bar{A}B\bar{C}\bar{D}$	$\bar{A}B\bar{C}D$	$\bar{A}BCD$	$\bar{A}BC\bar{D}$
AB	$AB\bar{C}\bar{D}$	$AB\bar{C}D$	$ABCD$	$ABC\bar{D}$
$A\bar{B}$	$A\bar{B}\bar{C}\bar{D}$	$A\bar{B}\bar{C}D$	$A\bar{B}CD$	$A\bar{B}C\bar{D}$

Minimization using Karnaugh Maps

AB \ CD	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	0	0	0	0
10	1	0	0	0

$$f = \sum(0,8) = \overline{B} \bullet \overline{C} \bullet \overline{D}$$

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	0	0
11	0	1	0	0
10	0	0	0	0

$$f = \sum(5,13) = B \bullet \overline{C} \bullet D$$

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	1	1	0
10	0	0	0	0

$$f = \sum(13,15) = A \bullet B \bullet D$$

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	0	0	1
11	0	0	0	0
10	0	0	0	0

$$f = \sum(4,6) = \overline{A} \bullet B \bullet \overline{D}$$

AB \ CD	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	0	0

$$f = \sum(2,3,6,7) = \overline{A} \bullet C$$

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	0	0	1
11	1	0	0	1
10	0	0	0	0

$$f = \sum(4,6,12,14) = B \bullet \overline{D}$$

AB \ CD	00	01	11	10
00	0	0	1	1
01	0	0	0	0
11	0	0	0	0
10	0	0	1	1

$$f = \sum(2,3,10,11) = \overline{B} \bullet C$$

AB \ CD	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	0	0	0
10	1	0	0	1

$$f = \sum(0,2,8,10) = \overline{B} \bullet \overline{D}$$

Minimization using Karnaugh Maps

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	0	0	0	0

$f = \sum(4,5,6,7) = \overline{A} \bullet B$

AB \ CD	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	0
10	0	0	1	0

$f = \sum(3,7,11,15) = C \bullet D$

AB \ CD	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

$f = \sum(0,3,5,6,9,10,12,15)$

$f = A \otimes B \otimes C \otimes D$

AB \ CD	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

$f = \sum(1,2,4,7,8,11,13,14)$

$f = A \oplus B \oplus C \oplus D$

AB \ CD	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

$f = \sum(1,3,5,7,9,11,13,15)$

$f = D$

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

$f = \sum(0,2,4,6,8,10,12,14)$

$f = \overline{D}$

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	1	1	1
10	0	0	0	0

$f = \sum(4,5,6,7,12,13,14,15)$

$f = B$

AB \ CD	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	0	0
10	1	1	1	1

$f = \sum(0,1,2,3,8,9,10,11)$

$f = \overline{B}$

Minimization using Karnaugh Maps

AB \ CD	00	01	11	10
00	1	1		1
01	1	1		1
11	1	1		1
10	1	1		

$$f = \overline{C} + \overline{A}\overline{D} + B\overline{D}$$

AB \ CD	00	01	11	10
00	1	1		1
01				1
11				
10	1	1		1

$$f = \overline{B}\overline{C} + \overline{B}\overline{D} + \overline{A}C\overline{D}$$

AB \ CD	00	01	11	10
00				
01	1	1	1	
11	1	1		1
10	1			

$$f = B\overline{C} + AB\overline{D} + A\overline{C}\overline{D} + \overline{A}BD$$

AB \ CD	00	01	11	10
00		1	1	
01	1	1	1	1
11	1		1	1
10			1	

$$f = \overline{A}D + B\overline{D} + CD$$

Minimization using Karnaugh Maps

Five-variable K-Map:

- Two input variables: A, B, C, D, E
- 32 output combinations (minterms):

$\bar{A}$

BC \ DE	$\bar{D}\bar{E}$	$\bar{D}E$	DE	$D\bar{E}$
$\bar{B}\bar{C}$	0	1	3	2
$\bar{B}C$	4	5	7	6
BC	12	13	15	14
$B\bar{C}$	8	9	11	10

A

BC \ DE	$\bar{D}\bar{E}$	$\bar{D}E$	DE	$D\bar{E}$
$\bar{B}\bar{C}$	16	17	19	18
$\bar{B}C$	20	21	23	22
BC	28	29	31	30
$B\bar{C}$	24	25	27	26

Minimization using Karnaugh Maps

Simplify $\boldsymbol{f(A,B,C,D,E) = \sum m(0,2,3,10,11,12,13,16,17,18,19,20,21,26,27)}$

$\bar{A}$

BC \ DE	$\bar{D}\bar{E}$	$\bar{D}E$	DE	$D\bar{E}$
$\bar{B}\bar{C}$	1		1	1
$\bar{B}C$				
BC	1	1		
$B\bar{C}$			1	1

A

BC \ DE	$\bar{D}\bar{E}$	$\bar{D}E$	DE	$D\bar{E}$
$\bar{B}\bar{C}$	1	1	1	1
$\bar{B}C$	1	1		
BC				
$B\bar{C}$			1	1

$$f = \bar{A}BC\bar{D} + \bar{B}\bar{C}\bar{E} + A\bar{B}\bar{D} + \bar{C}D$$

Minimization using Karnaugh Maps

Don't Care Conditions in K-Map:

- In BCD code, the binary states 1010, 1011, 1100, 1101, 1110, and 1111 are invalid; the corresponding outputs are don't cares
- The don't care states are denoted d, X, or Ø
- During SOP K-map, each don't care is treated as '1' if they are useful, otherwise considered as '0'

$$\textbf{Simplify} f(A,B,C,D) = \sum m(1,5,6,12,13,14) + d(2,4)$$

AB \ CD	00	01	11	10
00		1		X
01	X	1		1
11	1	1		1
10				

$$f = B\bar{C} + B\bar{D} + \bar{A}\bar{C}D$$

Minimization using Karnaugh Maps

K-Map of Maxterms:

Two-Variables

A \ B	0	1
0	$A+B$ (M_0)	$A+\bar{B}$ (M_1)
1	$\bar{A}+B$ (M_2)	$\bar{A}+\bar{B}$ (M_3)

Simplify $f(A,B) = \prod M(0,2,3)$

A \ B	B	$\bar{B}$
A	0 (0)	1 (1)
$\bar{A}$	0 (2)	0 (3)

$$f = (B)\cdot(\bar{A})$$

Three-Variables

A \ BC	00	01	11	10
0	$A+B+C$ (M_0)	$A+B+\bar{C}$ (M_1)	$A+\bar{B}+\bar{C}$ (M_3)	$A+\bar{B}+C$ (M_2)
1	$\bar{A}+B+C$ (M_4)	$\bar{A}+B+\bar{C}$ (M_5)	$\bar{A}+\bar{B}+\bar{C}$ (M_7)	$\bar{A}+\bar{B}+C$ (M_6)

Simplify $f(A,B,C) = \prod M(0,1,2,3,4,7)$

BC	00	01	11	10
0	0	0	0	0
1	0	1	0	1

$$f = (A)\cdot(B+C)\cdot(\bar{B}+\bar{C})$$

Minimization using Karnaugh Maps

Four-Variables

AB \ CD	00 $C+D$	01 $C+\bar{D}$	11 $\bar{C}+\bar{D}$	10 $\bar{C}+D$
00 $A+B$	$A+B+C+D$ (M_0)	$A+B+C+\bar{D}$ (M_1)	$A+B+\bar{C}+\bar{D}$ (M_3)	$A+B+\bar{C}+D$ (M_2)
01 $A+\bar{B}$	$A+\bar{B}+C+D$ (M_4)	$A+\bar{B}+C+\bar{D}$ (M_5)	$A+\bar{B}+\bar{C}+\bar{D}$ (M_7)	$A+\bar{B}+\bar{C}+D$ (M_6)
11 $\bar{A}+\bar{B}$	$\bar{A}+\bar{B}+C+D$ (M_{12})	$\bar{A}+\bar{B}+C+\bar{D}$ (M_{13})	$\bar{A}+\bar{B}+\bar{C}+\bar{D}$ (M_{15})	$\bar{A}+\bar{B}+\bar{C}+D$ (M_{14})
10 $\bar{A}+B$	$\bar{A}+B+C+D$ (M_8)	$\bar{A}+B+C+\bar{D}$ (M_9)	$\bar{A}+B+\bar{C}+\bar{D}$ (M_{11})	$\bar{A}+B+\bar{C}+D$ (M_{10})

Minimization using Karnaugh Maps

Four-Variables

AB \ CD	00 $C+D$	01 $C+\bar{D}$	11 $\bar{C}+\bar{D}$	10 $\bar{C}+D$
00 $A+B$	$A+B+C+D$ (M_0)	$A+B+C+\bar{D}$ (M_1)	$A+B+\bar{C}+\bar{D}$ (M_3)	$A+B+\bar{C}+D$ (M_2)
01 $A+\bar{B}$	$A+\bar{B}+C+D$ (M_4)	$A+\bar{B}+C+\bar{D}$ (M_5)	$A+\bar{B}+\bar{C}+\bar{D}$ (M_7)	$A+\bar{B}+\bar{C}+D$ (M_6)
11 $\bar{A}+\bar{B}$	$\bar{A}+\bar{B}+C+D$ (M_{12})	$\bar{A}+\bar{B}+C+\bar{D}$ (M_{13})	$\bar{A}+\bar{B}+\bar{C}+\bar{D}$ (M_{15})	$\bar{A}+\bar{B}+\bar{C}+D$ (M_{14})
10 $\bar{A}+B$	$\bar{A}+B+C+D$ (M_8)	$\bar{A}+B+C+\bar{D}$ (M_9)	$\bar{A}+B+\bar{C}+\bar{D}$ (M_{11})	$\bar{A}+B+\bar{C}+D$ (M_{10})

Simplify $\boldsymbol{f(A,B,C,D) = \prod M(2,8,9,10,11,12,14)}$

AB \ CD	00	01	11	10
00				0
01				
11	0			0
10	0	0	0	0

$$f = (\bar{A}+B)\cdot(\bar{A}+D)\cdot(B+\bar{C}+D)$$

Design of combinational digital circuits

- Steps to design a combinational digital circuit:
  - From the problem statement, derive the truth table
  - From the truth table, derive the Boolean expression
  - Simplify the logic expression
  - From the simplified expression, draw the logic circuit

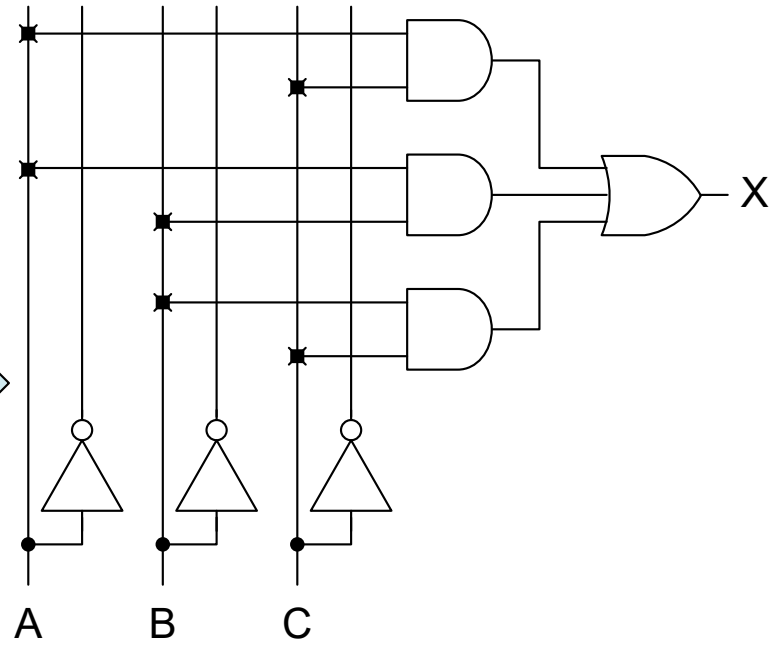
- **Design a 3-input (A, B, C) digital circuit that will give at its output (X) a logic 1 only if the binary number formed at the input has more ones than zeros.**

	Inputs A	B	C	Output X
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

$$X = \sum(3, 5, 6, 7)$$

A \ BC	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$X = AC + AB + BC$$

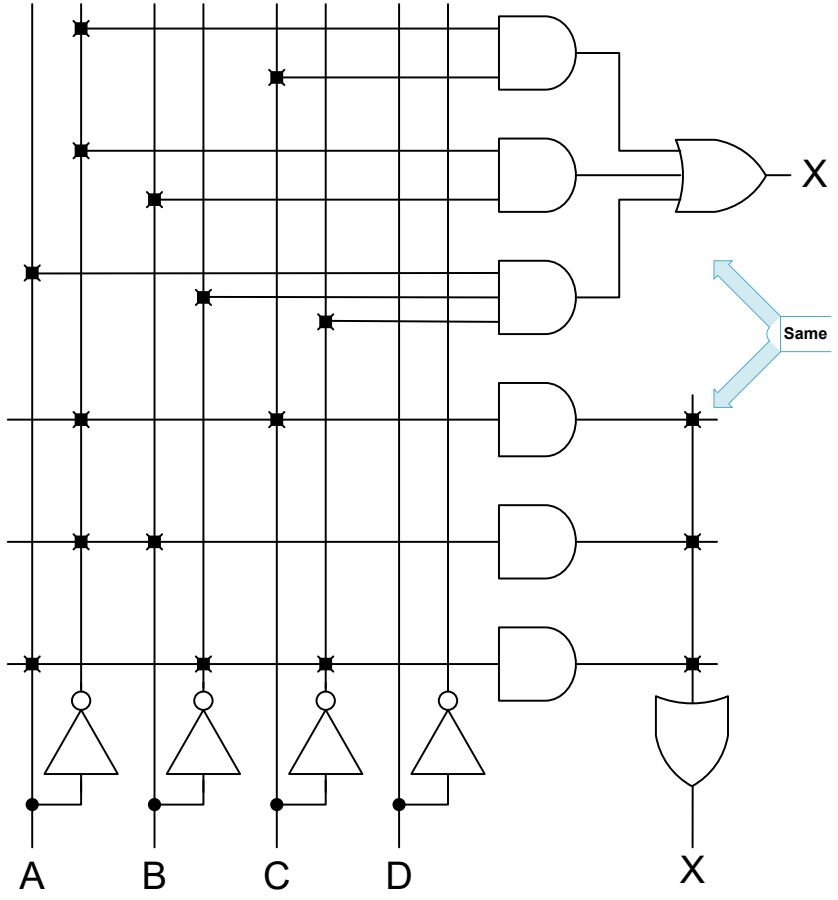


Design of combinational digital circuits

Design a 4-input (A,B,C,D) digital circuit that will give at its output (X) a logic 1 only if the binary number formed at the input is between 2 and 9 (including).

	Inputs				Output
	A	B	C	D	X
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	1
10	1	0	1	0	0
11	1	0	1	1	0
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	0
15	1	1	1	1	0

$$X = \sum(2,3,4,5,6,7,8,9)$$

AB \ CD	00	01	11	10
00	0	0	1	1
01	1	1	1	1
11	0	0	0	0
10	1	1	0	0

$$X = \overline{A}C + \overline{A}B + A\overline{B}\,\overline{C}$$